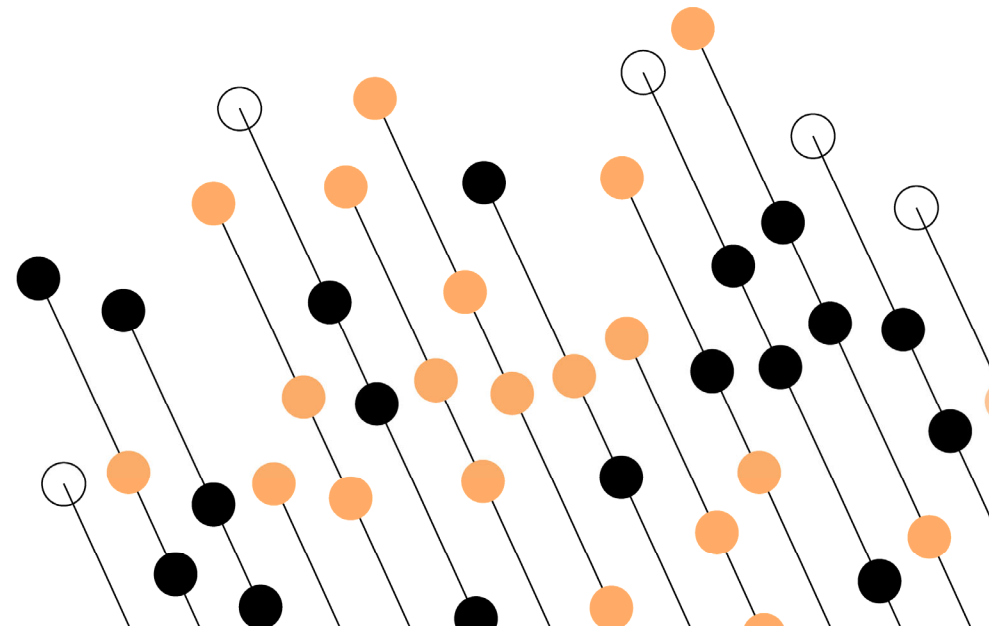


Polymorphism



Contents



Objectives

- To understand and use polymorphism.



Contents

- Constructors — how they are affected.
- Overriding of methods.
- Substitutability.
- Runtime method version look up — polymorphism.

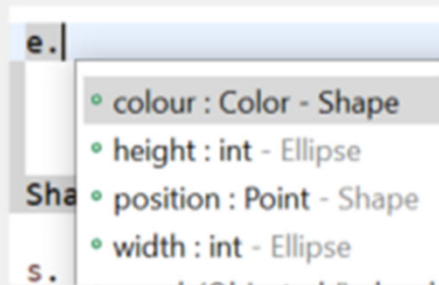


Hands-on labs

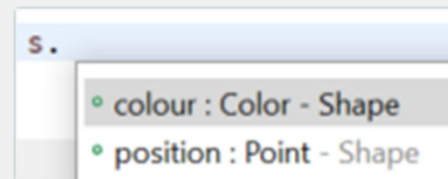
The principle of substitutability

- **Object of derived type exhibits all behaviour of base type.**
 - A derived object is a 'kind of' base object.

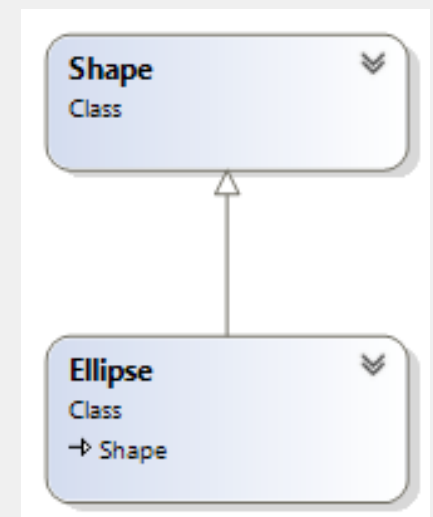
```
Ellipse e = new Ellipse();  
Shape s = e;
```



But why would you do it?



Missing all the Ellipses stuff



Why use substitution of references?

```
public static Shape makeShape(int picNo) {  
    if (picNo == 1)  
        return new Ellipse(10,5, 100,20);  
  
    if (picNo == 2)  
        return new Rectangle(3,10, 20,50);  
}
```

Returns a parameter of base class type (Shape)

```
public static void main(String[] args) {  
  
    Shape s = makeShape(1);  
  
    Ellipse e = (Ellipse) makeShape(1);  
}
```

Why use substitution of references? (2)

```
Shape[] shapes = {  
    myEllipse,  
    yourTriangle,  
    ourCircle  
};
```

Collections and arrays

```
for (Shape s : shapes)  
{  
    drawShape(s);  
}
```

```
public void drawShape(Shape shape) {  
    // code for drawing a shape  
}
```

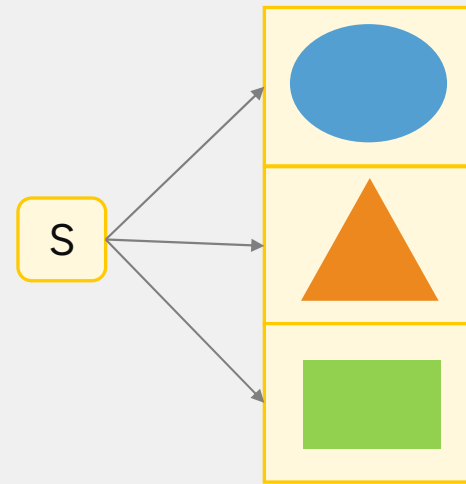
Is passed a parameter of base class type (Shape)

Towards polymorphism

- Morphing into many shapes

```
Shape[] shapes = {  
    myEllipse,  
    yourTriangle,  
    ourRectangle  
};  
  
for (Shape s : shapes) {  
    drawShape(s);  
}
```

```
public void drawShape(Shape shape) {  
    // code for drawing a shape  
}
```



Overriding base class methods

➔ `Rectangle rec = new Rectangle();`
`System.out.println(rec.getArea());`

```
class Shape {  
    public int getArea() {  
        return 0;  
    }  
}  
  
class Rectangle extends Shape {  
    public int getArea() {  
        return 100;  
    }  
}
```

which method *is invoked*?

Shape getArea() or
Rectangle getArea()

100

Overriding base class methods (2)

```
➔ Shape rec = new Rectangle();  
  
System.out.println(rec.getArea());
```

```
class Shape {  
    public int getArea() {  
        return 0;  
    }  
}  
  
class Rectangle extends Shape {  
    public int getArea() {  
        return 100;  
    }  
}
```

which method *is invoked*?

Shape getArea() or
Rectangle getArea()

100

Enabling overriding

- A derived class might want to alter implementation.
- It's best to use the **@Override** annotation.
 - Compiler checks the method and its parameters.
 - Good indication to the other developers.

```
public class Shape {  
    public Point position;  
    public Color colour;  
  
    public int getArea() {  
        return 0;  
    }  
}
```

```
public class Rectangle extends Shape {  
    public int width, height;  
  
    @Override  
    public int getArea() {  
        return width * height;  
    }  
}
```

Using **@Override**
is good practice. 👍

Polymorphism — Lists and arrays

```
public class Shape {  
    public int width, height;  
  
    public String colour;  
  
    public int getArea() {  
        return 0;  
    }  
}
```

```
public class Rectangle extends Shape {  
  
    public int getArea() {  
        return width * height;  
    }  
}
```

```
Shape myShape = new Shape();  
Rectangle myRectangle = new Rectangle();  
  
Shape[] shapes = {myShape, myRectangle};  
  
for (Shape s : shapes)  
    print(s.getArea());
```

Which of the getArea() methods are invoked? Shape or Rectangle?

Basics of casting — Down casting

- The data type of a reference that controls what is 'visible':

```
class Person {  
    private String name;  
    public String getName() {  
        return name;  
    }  
}
```

```
class Student extends Person {  
    private String subject;  
    public String getSubject() {  
        return subject;  
    }  
}
```

```
Person[] people = {new Person(), new Student()};  
for(Person p : people) {  
    print(p.getName());           // Every Person has a name  
    print(p.getSubject());        // Person has no subject  
  
    Student s = (Student)p;       // new reference has new type  
    print(s.getSubject());        // Student has Subject  
}
```

Compiles but will it crash at runtime?

Safe down casting

- A downcast could fail at runtime with '`ClassCastException`'.
- Test whether cast is safe via the `instanceof` keyword.



```
Person[] people = { new Person(), new Student() };

for(Person p : people) {
    System.out.print(p.getName());           // Every Person has a name

    if (p instanceof Student) {
        Student s = (Student) p;           // cast to Student type
        System.out.println(s.getSubject()); // Student has Subject
    }
}
```

Invoking base class functionality

- A derived class can access base class member.

Calls first method with matching signature up the inheritance hierarchy

```
public class Student extends Person {  
  
    private String subject;  
    public String getDetails() {  
  
        String data = super.getDetails(); // call base class  
  
        //..... code  
  
        return data + "\t" + subject;  
    }  
}
```

Non-extendible classes and methods

- A class can be written with the `final` modifier to prevent extension.

```
public final class String {  
    ...  
}
```

- A method can be marked `final` to stop it being overridden.

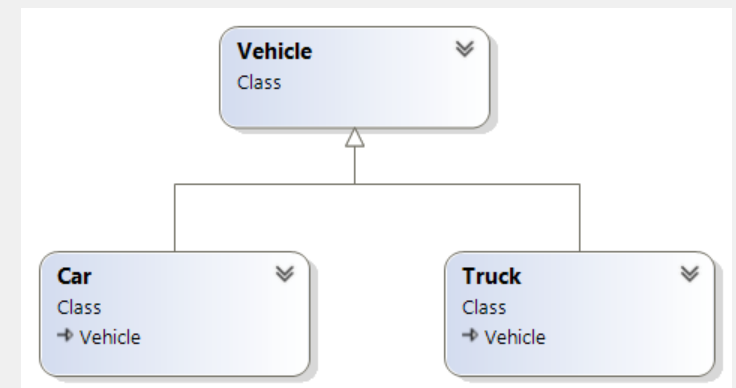


Sealed class — Introduced in Java 17

- Used to restrict which other classes or interfaces can extend or implement a class.
- Good for controlling the class hierarchy and preventing unintended inheritance.

```
sealed class Vehicle permits Car, Truck {  
    String type;  
  
    Vehicle(String type)  
    {  
        this.type = type;  
    }  
}
```

Only Car and Truck may extend **Vehicle**

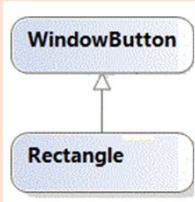


Best practice



Use inheritance only for genuine 'is a' relationships.

It's logical to substitute the object of derived class for the object of base class Methods in base class to make sense in derived class.



Ad-hoc inheritance for short-term convenience tends to lead to future problems and surprises!

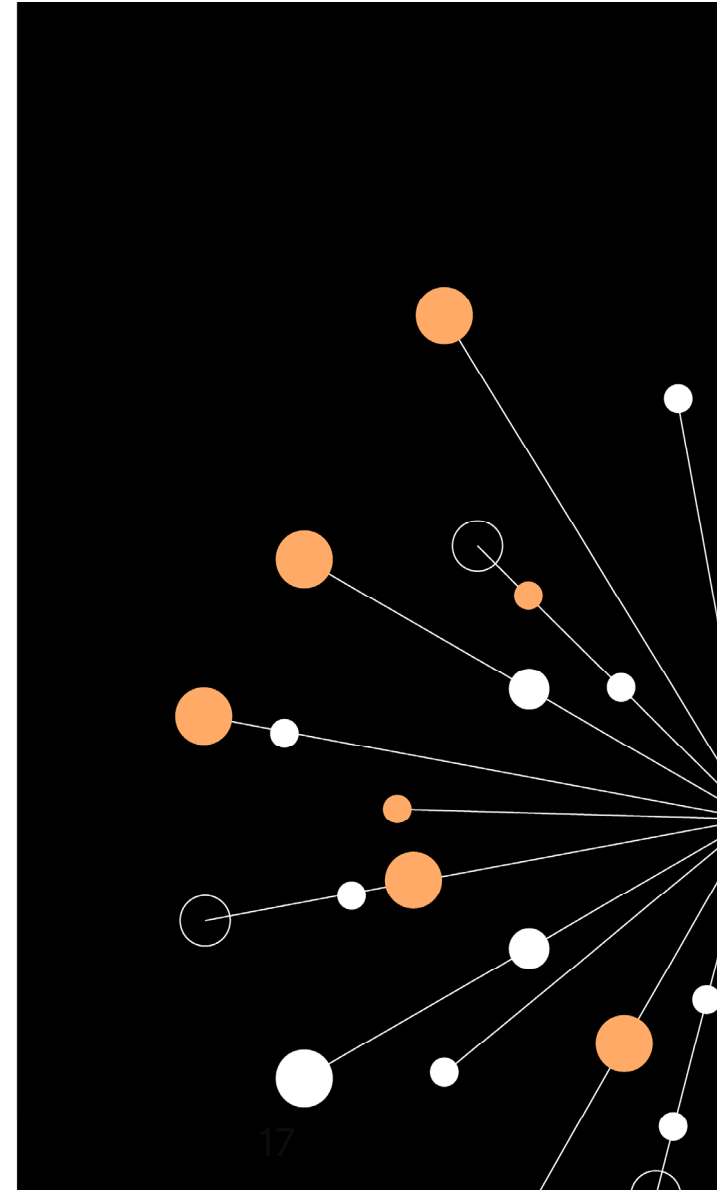


Java only supports single inheritance.

Choosing a base class is therefore significant.

In this chapter, we reviewed...

- Why do we do Inheritance?
 - We can upcast refs to a common base type to effect polymorphism.
 - Maybe a bit of code reuse as well.
- Derived class inherits and can override and add.
- Method calls automatically polymorphic.
- Started to look at (down)casting.



Lab

- Working with inheritance
 - Employee hierarchy

QA

